

Chapter 6: Process Synchronization

- Background
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
- Monitors
- Synchronization Examples
- Atomic Transactions

Background

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.

■ Shared-memory solution to bounded-buffer problem (Chapter 4) allows at most $n - 1$ items in buffer at the same time. A solution, where all N buffers are used is not simple.

☞ Suppose that we modify the producer-consumer code by adding a variable *counter*, initialized to 0 and incremented each time a new item is added to the buffer

Bounded-Buffer

- Shared data

```
#define BUFFER_SIZE 10
```

```
typedef struct {
```

```
    . . .
```

```
} item;
```

```
item buffer[BUFFER_SIZE];
```

```
int in = 0;int out = 0;int counter = 0;
```

Bounded-Buffer

■ Producer process

```
item nextProduced;  
while (1) {  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

Bounded-Buffer

■ Consumer process

```
item nextConsumed;  
while (1) {  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
}
```

Bounded Buffer

- The statements
counter++;
counter--;
must be performed *atomically*.
- Atomic operation means an operation that completes in its entirety without interruption.

Bounded Buffer

- The statement “**count++**” may be implemented in machine language as:
register1 = counter
register1 = register1 + 1
counter = register1

The statement “**count—**” may be implemented as:

register2 = counter

register2 = register2 – 1

counter = register2

Bounded Buffer

- If both the producer and consumer attempt to update the buffer concurrently, the assembly language statements may get interleaved.
- Interleaving depends upon how the producer and consumer processes are scheduled.

Bounded Buffer

- Assume **counter** is initially 5. One interleaving of statements is:
producer: **register1 = counter** (*register1 = 5*)
producer: **register1 = register1 + 1** (*register1 = 6*)
consumer: **register2 = counter** (*register2 = 5*)

consumer: **register2** = **register2** – 1
(*register2* = 4)

producer: **counter** = **register1** (*counter* =
6)

consumer: **counter** = **register2** (*counter* =
4)

The value of **count** may be either 4 or 6,
where the correct result should be 5.

Consider this execution interleaving with
“count = 5” initially:

S0: producer execute $\text{register1} = \text{count}$
{register1 = 5}

S1: producer execute $\text{register1} = \text{register1} + 1$
{register1 = 6}

S2: consumer execute $\text{register2} = \text{count}$
{register2 = 5}

S3: consumer execute $\text{register2} = \text{register2} - 1$
{register2 = 4}

S4: producer execute $\text{count} = \text{register1}$
{count = 6 }

S5: consumer execute $\text{count} = \text{register2}$
{count = 4}

Peterson's Solution

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
 - ☞ `int turn;`
 - ☞ `Boolean flag[2]`
- The variable `turn` indicates whose turn it is to enter the critical section.
- The `flag` array is used to indicate if a process is ready to enter the critical section. `flag[i] = true` implies that process P_i is ready!

Algorithm for Process P_i

```
while (true) {  
    flag[i] = TRUE;  
    turn = j;  
    while ( flag[j] && turn == j);
```

CRITICAL SECTION

```
    flag[i] = FALSE;
```

REMAINDER SECTION

```
}
```

Process P_j

```
while (true) {  
    flag[j] = TRUE;  
    turn = i;  
    while ( flag[i] && turn == i);
```

CRITICAL SECTION

```
    flag[j] = FALSE;
```

REMAINDER SECTION

```
}
```


Synchronization Hardware

- Many systems provide hardware support for critical section code
- Uniprocessors – could disable interrupts
 - ☞ Currently running code would execute without preemption
 - ☞ Generally too inefficient on multiprocessor systems
- Modern machines provide special atomic hardware instructions
 - ☞ Either test memory word and set value
 - ☞ Or swap contents of two memory words

TestAndndSet Instruction

■ Definition:

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

Solution using TestAndSet

- Shared boolean variable lock., initialized to false.
- Solution:

```
while (true) {  
    while ( TestAndSet (&lock ))  
        ; /* do nothing  
  
        //    critical section  
  
    lock = FALSE;  
  
        //    remainder section  
  
}
```

Swap Instruction

■ Definition:

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```

Solution using Swap

- Shared Boolean variable lock initialized to FALSE; Each process has a local Boolean variable key.
- Solution:

```
while (true) {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
  
        //    critical section  
  
    lock = FALSE;  
  
        //    remainder section  
  
}
```

Semaphores

- Synchronization tool that does not require busy waiting.
- Semaphore S – integer variable
- can only be accessed via two indivisible (atomic) operations

wait (*S*):

while $S \leq 0$ do *no-op*;
 $S--$;

signal (*S*):

$S++$;

Critical Section of n Processes

■ Shared data:

semaphore mutex; //initially *mutex* = 1

■ Process P_i :

```
do {  
    wait(mutex);  
    critical section  
    signal(mutex);  
    remainder section  
} while (1);
```

Semaphore Implementation

- Define a semaphore as a record

```
typedef struct {  
    int value;  
    struct process *L;  
} semaphore;
```

- Assume two simple operations:
 - ➡ **block** suspends the process that invokes it.
 - ➡ **wakeup(*P*)** resumes the execution of a blocked process **P**.

Implementation

- Semaphore operations now defined as

wait(S):

S.value--;

if (S.value < 0) {

add this process to S.L;

block;

}

```
signal(S):  
    S.value++;  
    if (S.value <= 0) {  
        remove a process P from S.L;  
        wakeup(P);  
    }
```

Semaphore as a General Synchronization Tool

- Execute B in P_j only after A executed in P_i
- Use semaphore *flag* initialized to 0
- Code:

P_i	P_j
$\vdots$	$\vdots$
A	$wait(flag)$
$signal(flag)$	B

Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes.
- Let S and Q be two semaphores initialized to 1.

P_0

wait(S);

wait(Q);

⋮

signal(S);

signal(Q)

P_1 **f**

wait(Q);

wait(S);

signal(Q);

signal(S);

- **Starvation** – indefinite blocking. A process may never be removed from the semaphore queue in which it is suspended

Two Types of Semaphores

- *Counting* semaphore – integer value can range over an unrestricted domain.
- *Binary* semaphore – integer value can range only between 0 and 1; can be simpler to implement.
- Can implement a counting semaphore S as a binary semaphore.

Implementing S as a Binary Semaphore

■ Data structures:

binary-semaphore S1, S2;

int C;

■ Initialization:

S1 = 1

S2 = 0

C = initial value of semaphore S

Implementing S

■ *wait* operation

```
wait(S1);  
C--;  
if (C < 0) {  
    signal(S1);  
    wait(S2);  
}  
signal(S1);
```

■ *signal* operation

wait(S1);

C ++;

if (C <= 0)

signal(S2);

else

signal(S1);

Classical Problems of Synchronization

- Bounded-Buffer Problem
- Readers and Writers Problem
- Dining-Philosophers Problem

Bounded-Buffer Problem

■ Shared data

**semaphore full, empty,
mutex;**

Initially:

full = 0, empty = n, mutex = 1

Bounded-Buffer Problem Producer Process

```
do {  
    ...  
    produce an item in nextp  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
}
```

add **nextp** to buffer

...

signal(mutex);

signal(full);

} while (1);

Bounded-Buffer Problem Consumer Process

```
do {  
    wait(full)  
    wait(mutex);  
    ...  
    remove an item from buffer to  
nextc  
    ...
```

```
signal(mutex);
```

```
    signal(empty);
```

```
    ...
```

```
    consume the item in nextc
```

```
    ...
```

```
} while (1);
```

Readers-Writers Problem

■ Shared data

semaphore mutex, wrt;

Initially

**mutex = 1, wrt = 1, readcount
= 0**

Readers-Writers Problem Writer Process

wait(wrt);

...

writing is performed

...

signal(wrt);

Readers-Writers Problem Reader Process

```
wait(mutex);  
readcount++;  
if (readcount == 1)  
    wait(wrt);  
signal(mutex);  
...
```

reading is performed

...

wait(mutex);

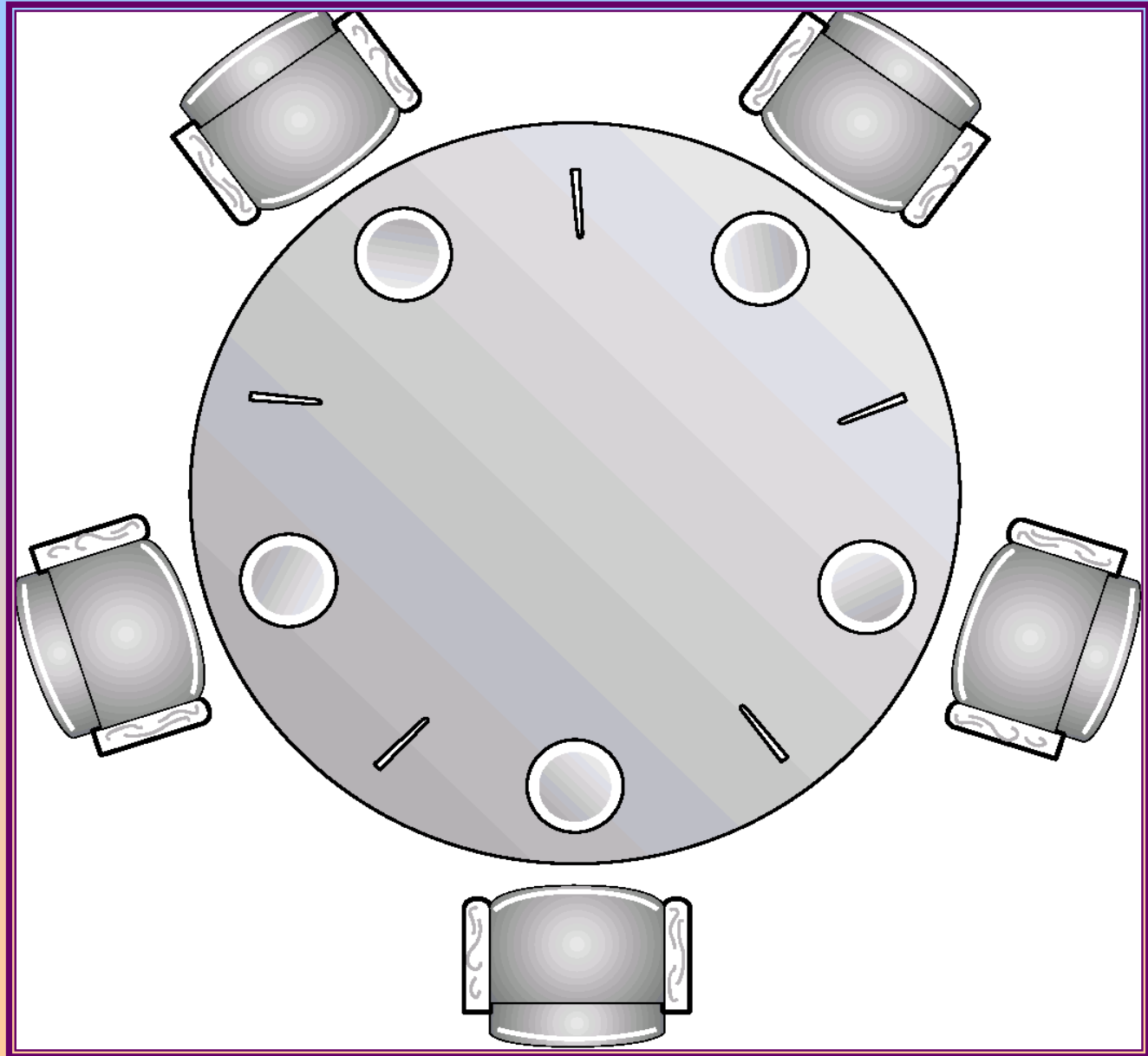
readcount--;

if (readcount == 0)

signal(wrt);

signal(mutex):

Dining-Philosophers Problem



- Shared data

semaphore chopstick[5];

Initially all values are 1

Dining-Philosophers Problem

■ Philosopher i :

do {

wait(chopstick[i])

wait(chopstick[(i+1) % 5])

...

eat

...

```
signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
    ...  
    think  
    ...  
} while (1);
```

Critical Regions

- High-level synchronization construct
- A shared variable $\mathbf{v}$ of type $\mathbf{T}$, is declared as:

$\mathbf{v}$: shared $\mathbf{T}$

- Variable ***v*** accessed only inside statement

region *v* when *B* do *S*

where ***B*** is a boolean expression.

- While statement ***S*** is being executed, no other process can access variable ***v***.

Critical Regions

- Regions referring to the same shared variable exclude each other in time.
- When a process tries to execute the region statement, the Boolean expression B is evaluated. If B is true, statement S is executed. If it is false, the process is delayed until B becomes true and no other process is in the region associated with v .

Example – Bounded Buffer

■ Shared data:

```
struct buffer {  
    int pool[n];  
    int count, in, out;  
}
```

Bounded Buffer Producer Process

- Producer process inserts **nextp** into the shared buffer

```
region buffer when( count < n) {  
    pool[in] = nextp;  
    in := (in+1) % n;  
    count++;  
}
```

Bounded Buffer Consumer Process

- Consumer process removes an item from the shared buffer and puts it in **nextc**

```
region buffer when (count > 0) {  
    nextc = pool[out];  
    out = (out+1) % n;  
    count--;  
}
```


Implementation region x when B do S

- Associate with the shared variable x , the following variables:

**semaphore mutex, first-delay,
second-delay;**

int first-count, second-count;

- Mutually exclusive access to the critical section is provided by **mutex**.
- If a process cannot enter the critical section because the Boolean expression **B** is false, it initially waits on the **first-delay** semaphore; moved to the **second-delay** semaphore before it is allowed to reevaluate *B*.

Implementation

- Keep track of the number of processes waiting on **first-delay** and **second-delay**, with **first-count** and **second-count** respectively.
- The algorithm assumes a FIFO ordering in the queuing of processes for a semaphore.

- For an arbitrary queuing discipline, a more complicated implementation is required.

Monitors

- High-level synchronization construct that allows the safe sharing of an abstract data type among concurrent processes.

monitor *monitor-name*

{

shared variable declarations

procedure body *P1* (...) {

...

}

procedure body P_2 (...) {

...

}

procedure body P_n (...) {

...

}

{

initialization code

}

}

Monitors

- To allow a process to wait within the monitor, a **condition** variable must be declared, as

condition x, y;

- Condition variable can only be used with the operations **wait** and **signal**.

➡ The operation

x.wait();

means that the process invoking this operation is suspended until another process invokes

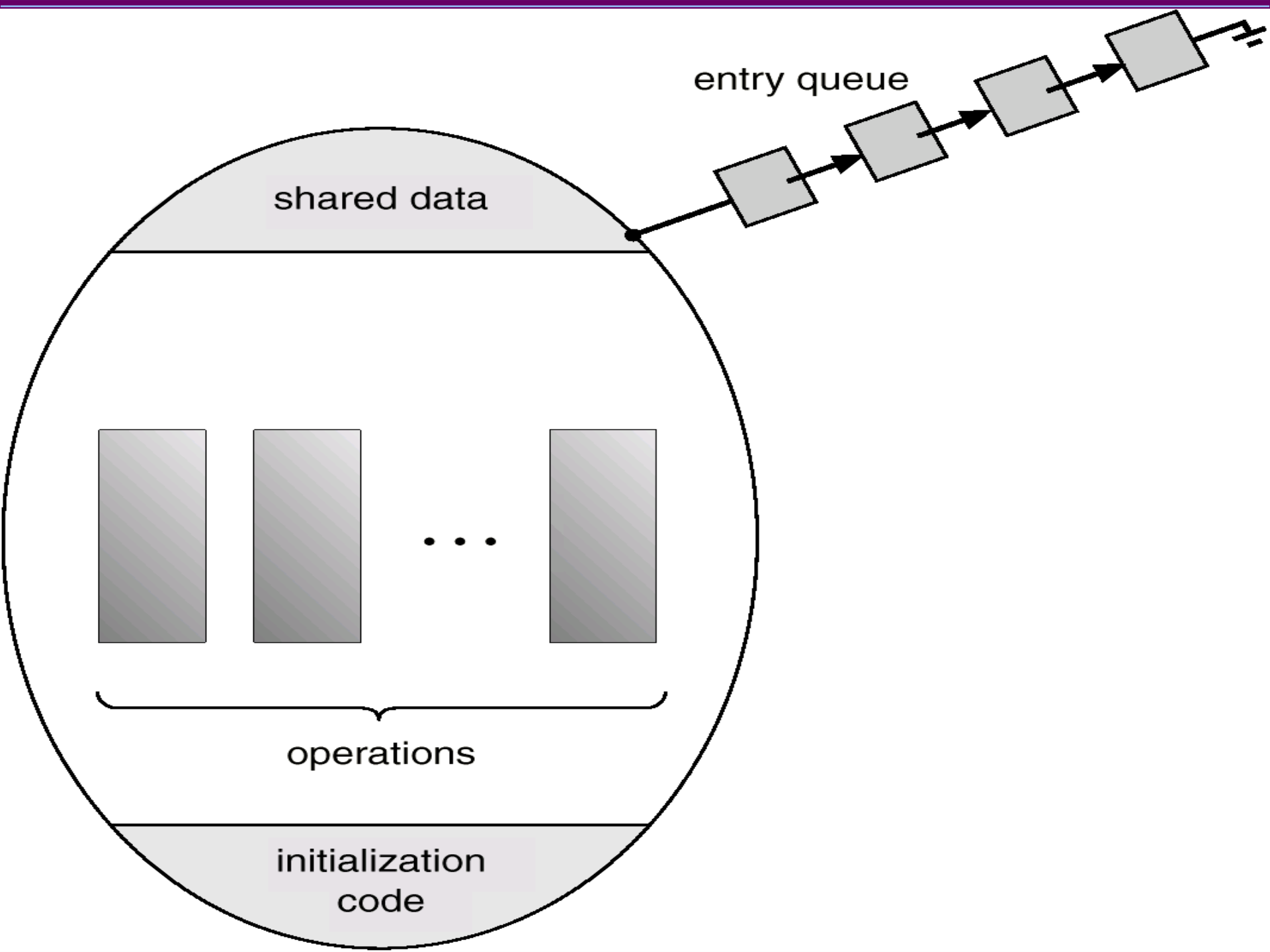
x.signal();

➡ The **x.signal** operation resumes exactly one suspended process. If no process is suspended, then the **signal** operation has no effect.

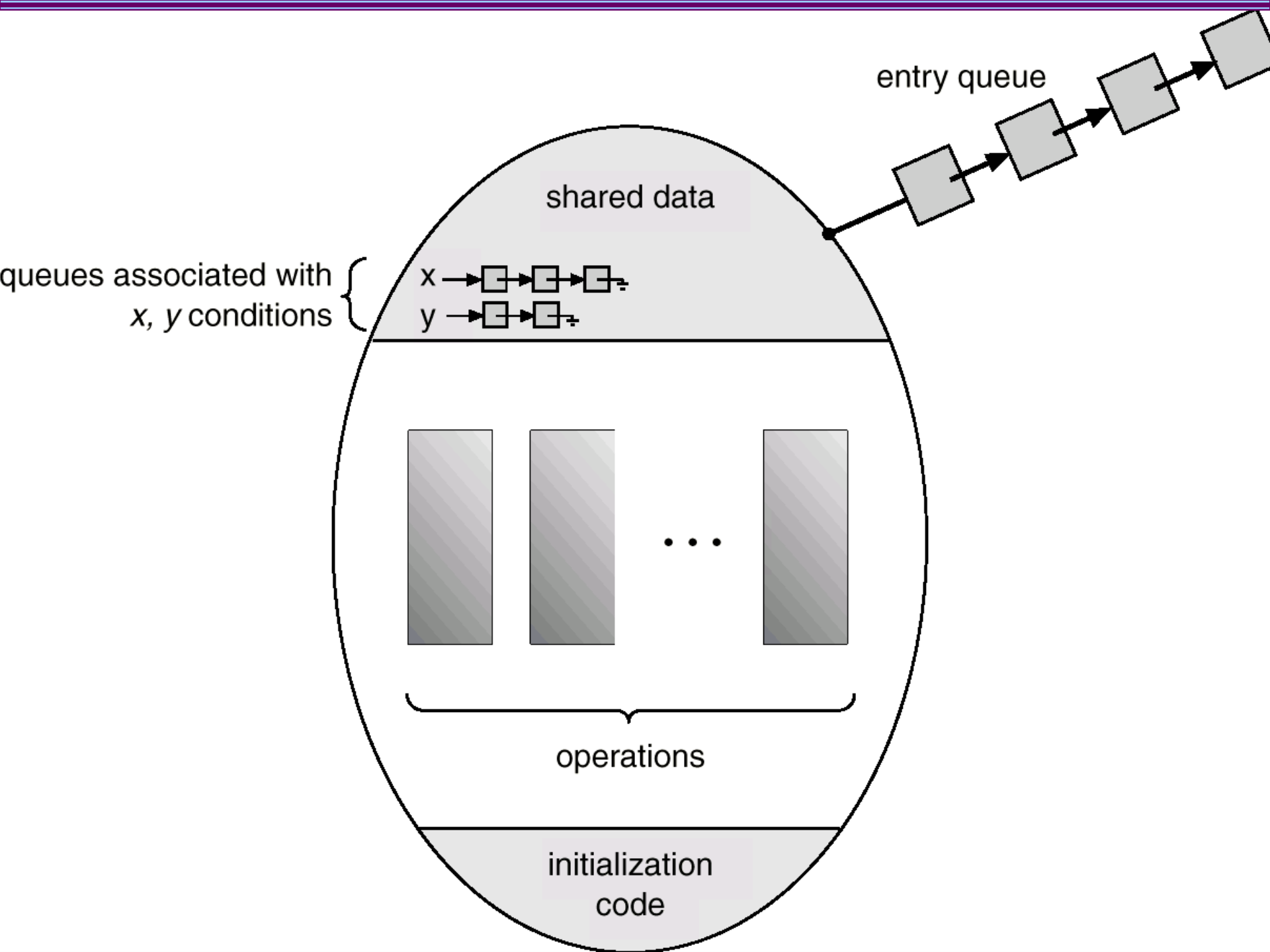
Schematic View of a Monitor

9

10



Monitor With Condition Variables



Dining Philosophers Example

monitor dp

{

enum {thinking, hungry, eating} state[5];

condition self[5];

void pickup(int i)

// following slides

```
void putdown(int i) // following slides
void test(int i)    // following slides
void init() {
    for (int i = 0; i < 5; i++)
        state[i] = thinking;
}
```

Dining Philosophers

```
void pickup(int i) {  
    state[i] = hungry;  
    test[i];  
    if (state[i] != eating)  
        self[i].wait();  
}
```

```
void putdown(int i) {  
    state[i] = thinking;  
    // test left and right neighbors  
    test((i+4) % 5);  
    test((i+1) % 5);  
}
```


Dining Philosophers

```
void test(int i) {  
    if ( (state[(i + 4) % 5] != eating) &&  
        (state[i] == hungry) &&  
        (state[(i + 1) % 5] != eating)) {  
        state[i] = eating;  
        self[i].signal();  
    }  
}
```

Monitor Implementation Using Semaphores

■ Variables

```
semaphore mutex; // (initially = 1)
semaphore next;  // (initially = 0)
int next-count = 0;
```

- Each external procedure *F* will be replaced by

```
wait(mutex);
```

```
...
```

```
body of  $F$ ;
```

```
...
```

```
if (next-count > 0)
```

```
    signal(next)
```

```
else
```

```
    signal(mutex);
```

- Mutual exclusion within a monitor is ensured.

Monitor Implementation

- For each condition variable **x**, we have:
semaphore x-sem; // (initially = 0)
int x-count = 0;
- The operation **x.wait** can be implemented as:

```
x-count++;  
    if (next-count > 0)  
        signal(next);  
else  
    signal(mutex);  
wait(x-sem);  
x-count--;
```

Monitor Implementation

- The operation **x.signal** can be implemented as:

```
if (x-count > 0) {  
    next-count++;  
    signal(x-sem);  
    wait(next);  
    next-count--;  
}
```

Monitor Implementation

■ *Conditional-wait* construct: **x.wait(c);**

- ☞ **c** – integer expression evaluated when the **wait** operation is executed.
- ☞ value of **c** (a *priority number*) stored with the name of the process that is suspended.
- ☞ when **x.signal** is executed, process with smallest associated priority number is resumed next.

■ Check two conditions to establish correctness of system:

- ☞ User processes must always make their calls on the monitor in a correct sequence.
- ☞ Must ensure that an uncooperative process does not ignore the mutual-exclusion gateway provided by the monitor, and try to access the shared resource directly, without using the access protocols.

Solaris Synchronization

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing.
- Uses *adaptive mutexes* for efficiency when protecting data from short code segments.

- Uses *condition variables* and *readers-writers* locks when longer sections of code need access to data.
- Uses *turnstiles* to order the list of threads waiting to acquire either an adaptive mutex or reader-writer lock.

Windows Synchronization

- Uses interrupt masks to protect access to global resources on uniprocessor systems.
- Uses *spinlocks* on multiprocessor systems.

- Also provides *dispatcher objects* which may act as wither mutexes and semaphores.
- Dispatcher objects may also provide *events*. An event acts much like a condition variable.

Windows Synchronization

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
- Also provides **dispatcher objects** which may act as either mutexes and semaphores
- Dispatcher objects may also provide **events**
 - ☞ An event acts much like a condition variable